

Zackery Fite

CS-300

Southern New Hampshire University

Professor Phillip Grabowski

February 15<sup>th</sup>, 2026

## **Module 5 Milestone Pseudocode**

### **Define Struct: Course**

courseID

courseName

preCount

preList

Constructor:

Set courseID = ""

Set courseName = ""

Set preCount = 0

Set preList = ""

### **Define Class: BinaryTree**

Struct Node

Course data

left pointer

right pointer

root pointer

Methods:

printCourse(courseID)

Constructor: BinaryTree()

### **Main Procedure**

Create a new BinaryTree called courseTree

Get CSV file path from user; if no path is provided, use default location

Call txtParser(filePath) to parse CSV into courseTree

Call validateList(courseTree) to ensure prerequisites are valid

Prompt user for a courseID to search and store in userSearch

Call printCourse(userSearch) to display course information

### **TxtParser(String)**

Function txtParser(filePath)

Open CSV file at filePath

Loop through each row until end of file

If first and second columns are present  
Add first column to courseID  
Add second column to courseName  
Initialize preCount = 0 and preNames = ""  
Loop through remaining columns until empty  
Increment preCount for each prerequisite  
Append prerequisite to preNames  
Store preCount in Course struct  
Store preNames in Course struct  
Return list of courses (tempList)

### **SearchList(String)**

Function searchList(courseID)  
Create tempCourse of type Node  
Set tempCourse to the bucket at hash(courseID)  
Loop through courses in the bucket  
If courseID matches, set tempCourse to that course  
Return tempCourse

### **Print Course(string)**

Function printCourse(courseID)  
Set tempCourse = root of BinaryTree  
While tempCourse is not null

If tempCourse courseID equals input courseID

Output courseID and courseName

Loop through each prerequisite in preList

Call printCourse(prerequisite) recursively

Else if courseID < input courseID, move tempCourse to left node

Else if courseID > input courseID, move tempCourse to right node

### **ValidateList**

Function validateList(courseTree)

Set valid = True

For each course in tree

If valid = False, break

For each prerequisite in preList

Set tempCourse = searchList(prerequisite)

If tempCourse courseID is empty, set valid = False

Return valid